

Client Layer

1. Customer App

Є клієнтським застосунком, через який користувач шукає ресторани та страви, створює і оплачує замовлення, переглядає історію та залишає оцінки. Для звичайних операцій застосунком взаємодіє з API GW через HTTPS/REST, а для отримання оновлень у реальному часі використовує WebSocket-з'єднання з Realtime GW. Статичний медіаконтент завантажується через CDN, а відображення карти виконується за допомогою *Maps Provider*. Автентифікація користувача виконується через *Identity Provider*, після чого отриманий JWT використовується у подальших запитах. Компонент може бути реалізований як мобільний застосунок на Flutter, React Native або нативних Android/iOS технологіях. Такий підхід дозволяє відокремити клієнтський інтерфейс від серверної бізнес-логіки та незалежно розвивати frontend і backend.

2. Courier App

Використовується кур'єром для отримання доступних замовлень, прийняття доставки та передачі своєї поточної геолокації. Синхронні операції, наприклад прийняття замовлення, виконуються через HTTPS/REST та API GW. Real-time повідомлення про доступні доставки і передача координат здійснюються через постійне WebSocket-з'єднання з Realtime GW. Автентифікація виконується через *Identity Provider*, а JWT дозволяє серверу ідентифікувати конкретного кур'єра. Компонент може бути реалізований як мобільний застосунок, оскільки він повинен мати доступ до GPS пристрою та працювати під час виконання доставки. Використання WebSocket замість частого HTTP polling зменшує кількість запитів і дозволяє підтримувати необхідну real-time поведінку.

3. Restaurant Portal

Є веб-застосунком для представників ресторанів, через який вони керують профілем закладу, меню, цінами та переглядають активні й історичні замовлення. CRUD-запити надсилаються через HTTPS/REST до API GW, а real-time координати активних кур'єрів надходять через Realtime GW по WebSocket. Для завантаження фотографій портал отримує дозвіл на upload і передає файли безпосередньо в *Object Storage* через presigned URL. Зображення для перегляду отримуються через CDN, а карта відображається за допомогою *Maps Provider*. Компонент може бути реалізований як SPA на React, Angular або Vue. Веб-портал є доцільним рішенням, оскільки представники ресторанів зазвичай працюють із системою через браузер і не потребують окремого нативного застосунку.

External Systems

1. Identity Provider / Auth Service

Відповідає за аутентифікацію користувачів, кур'єрів і представників ресторанів та видачу access token. Після успішного входу клієнт отримує JWT, який передає до API GW та Realtime GW. Gateway-компоненти перевіряють підпис токена та використовують його claims для визначення користувача і його ролі. Компонент може бути реалізований за допомогою Keycloak, Auth0, AWS Cognito або іншого OAuth 2.0/OpenID Connect провайдера. Централізована система ідентифікації дозволяє не дублювати логіку login, token management і role-based access control у кожному мікросервісі. Це особливо важливо для обмеження доступу, наприклад щоб представник ресторану міг змінювати лише власний заклад.

2. Payment Provider

Є зовнішньою платіжною системою, яка фактично виконує списання коштів з користувача. *Payment Service* взаємодіє з нею через захищений HTTPS API, а зміни статусу платежу можуть повертатися через webhook. Як реалізація можуть використовуватися Stripe, Aduyen або інший сертифікований платіжний провайдер. Дані банківських карток при цьому не повинні зберігатися безпосередньо сервісами системи. Використання зовнішнього провайдера спрощує роботу з PCI DSS, fraud detection та інтеграцією з різними платіжними методами. Це також дозволяє основній системі зосередитися на стані платежу та бізнес-логіці замовлення, а не на реалізації банківських транзакцій.

3. Maps Provider

Надає клієнтським застосункам карту, map tiles та SDK для відображення географічних об'єктів. *Customer App* і *Restaurant Portal* використовують його для візуалізації координат, отриманих від системи через Realtime GW. Компонент не відповідає за зберігання чи трансляцію координат кур'єрів, він лише відображає їх на карті. Реалізацією може бути Google Maps, Mapbox або OpenStreetMap-compatible provider. Використання готового картографічного сервісу усуває необхідність самостійно зберігати та обробляти великий набір map tiles і географічних даних. Це суттєво спрощує реалізацію функції real-time tracking.

Edge Layer

1. API GW

Є єдиною точкою входу для синхронних HTTP-запитів від усіх клієнтських застосунків. Він перевіряє JWT, виконує routing запитів до відповідних backend-сервісів, а також може реалізовувати rate limiting, request validation і TLS termination. Взаємодія клієнтів з Gateway виконується через HTTPS/REST, тоді як внутрішні виклики можуть використовувати REST або gRPC. Компонент може бути реалізований на базі Nginx, Envoy, Kong або managed cloud API GW. Централізований Gateway приховує внутрішню структуру мікросервісів від клієнтів та дозволяє змінювати backend без зміни зовнішнього API. Rate limiting і централізована перевірка доступу також допомагають захищати систему під час пікових навантажень.

2. Realtime Gateway

Підтримує довготривалі WebSocket-з'єднання з *Customer App*, *Courier App* та *Restaurant Portal*. Він отримує real-time події з Redis Pub/Sub та Kafka і маршрутизує їх лише тим клієнтам, які підписані на відповідне замовлення, ресторан або courier channel. Вхідні координати від кур'єрів також проходять через цей Gateway та передаються до *Location Service*. Для встановлення WebSocket-з'єднання перевіряється JWT, тому клієнт не може підписатися на чужі дані без необхідних прав. Компонент може бути реалізований на Go, Node.js або іншій event-driven платформі та горизонтально масштабуватися. WebSocket використовується замість частого polling, щоб забезпечити оновлення координат у межах вимоги 5 секунд при великій кількості одночасних підключень.

3. CDN

Використовується для швидкої доставки фотографій страв, логотипів та іншого статичного контенту. Клієнтські застосунки отримують файли безпосередньо з найближчого edge-вузла CDN, не навантажуючи основні backend-сервіси. Якщо необхідного файлу немає в кеші, CDN отримує його з *Object Storage*, який виступає origin. Компонент може бути реалізований за допомогою CloudFront, Cloudflare CDN або аналогічного сервісу. CDN добре підходить для системи з мільйонами користувачів, оскільки один і той самий контент часто завантажується багато разів. Географічне кешування також зменшує latency для кінцевих користувачів.

4. Object Storage

Є довготривалим сховищем для фотографій ресторанів, страв та іншого медіаконтенту. CDN використовує його як origin, а *Restaurant Portal* може завантажувати файли напряму за допомогою обмежених у часі presigned URL. *Restaurant & Catalog Service* контролює створення таких URL і зберігає лише посилання або metadata файлів у своїй базі. Компонент може бути реалізований за допомогою AWS S3, Google Cloud Storage або іншого object storage. *Object Storage* краще підходить для великих бінарних файлів, ніж реляційна база даних, та практично необмежено масштабується. Пряме завантаження файлів також зменшує network і CPU навантаження на application servers.

Core Services

1. Restaurant & Catalog Service

Керує профілями ресторанів, меню, категоріями, стравами, описами та цінами. CRUD-запити надходять через API GW, а дані зберігаються в окремій PostgreSQL базі. *Order Service* синхронно звертається до цього сервісу для перевірки актуальності страв і цін перед створенням замовлення. Після зміни ресторану чи меню сервіс публікує domain events, наприклад *RestaurantUpdated* або *MenuItemUpdated*, у Kafka для оновлення search index. Сервіс може бути реалізований як stateless REST/gRPC application на Java, Go, Python або іншій серверній платформі. Реляційна модель добре підходить для меню через наявність зв'язків між рестораном, категоріями, стравами, опціями та цінами і необхідність транзакційної цілісності.

2. Search Service

Відповідає за швидкий пошук ресторанів та страв за назвою, ключовими словами, категорією та іншими фільтрами. Користувачькі запити надходять через API GW, а читання виконується зі спеціалізованого Elasticsearch індексу. Актуальність індексу підтримується асинхронно: сервіс отримує *RestaurantUpdated* та *MenuItemUpdated* events з Kafka і оновлює відповідні документи. Сам сервіс може бути реалізований як stateless backend application, тоді як Elasticsearch виконує основну search-роботу. Спеціалізований search engine дозволяє ефективно виконувати full-text search, fuzzy matching, relevance ranking та складну фільтрацію. Відокремлення search workload від основної PostgreSQL бази також дозволяє масштабувати пошук незалежно від transactional operations.

3. Order Service

Керує основним життєвим циклом замовлення та є власником його транзакційного стану. Він отримує REST-запити через API GW, перевіряє меню і ціни через *Restaurant & Catalog Service* та ініціює оплату через *Payment Service*. Стан замовлення зберігається в *Orders PostgreSQL*, а важливі зміни стану публікуються у Kafka як domain events. Після переходу замовлення до стану, в якому потрібна доставка, сервіс публікує *DeliveryRequested*, а після завершення - *OrderCompleted*. Сервіс може бути реалізований як stateless application з реляційною базою даних та транзакційною state-machine логікою. PostgreSQL використовується тому, що зміни стану замовлення потребують надійних ACID-транзакцій та захисту від неконсистентних переходів.

4. Payment Service

Ізолює всю логіку, пов'язану з оплатою замовлення та станами платежу. *Order Service* викликає його синхронно через внутрішній REST або gRPC API, після чого сервіс взаємодіє із зовнішнім *Payment Provider* через HTTPS. Поточний стан платежів та idempotency keys зберігаються в окремому *Payments PostgreSQL*, а асинхронні зміни статусу можуть приходити від провайдера через webhook. Сервіс повинен підтримувати повторні запити без подвійного списання коштів та коректно обробляти timeout і retry сценарії. Реалізацією може бути stateless backend-сервіс з

PostgreSQL та інтеграцією з API платіжного провайдера. Виділення платежів в окремий компонент зменшує зв'язаність *Order Service* із зовнішньою фінансовою системою та дозволяє окремо забезпечувати підвищені вимоги до надійності.

5. Delivery / Dispatch Service

Відповідає за пошук доступних кур'єрів поблизу ресторану, надсилання їм delivery offers та фіксацію вибору кур'єра. Після отримання *DeliveryRequested* з Kafka сервіс виконує геопросторовий пошук у *Redis Cluster* та формує список відповідних кур'єрів. Пропозиції доставки публікуються як *DeliveryOffer* і через Realtime GW передаються кур'єрам. Коли кур'єр приймає замовлення через REST API, сервіс атомарно створює assignment у *Delivery PostgreSQL*, тому одне замовлення не може бути одночасно захоплене двома кур'єрами. Компонент може бути реалізований як stateless сервіс з PostgreSQL для durable state та Redis для швидкого geospatial lookup. Таке розділення дозволяє одночасно отримати низьку latency під час пошуку кур'єрів і надійність при фіксації фактичного призначення.

6. Location Service

Обробляє безперервний потік поточних координат активних кур'єрів. Координати надходять від *Courier App* через Realtime GW, після чого сервіс оновлює поточний стан кур'єра у *Redis Cluster*. Для миттєвої передачі координат клієнтам сервіс публікує *CourierLocationUpdated* у *Redis Pub/Sub*, звідки подія повертається до Realtime GW. Сервіс не зберігає кожен GPS-точку як критичну транзакційну інформацію, оскільки для карти найбільшу цінність має саме найновіша позиція. Компонент може бути реалізований як горизонтально масштабований stateless service, а Redis використовується як in-memory current-state storage. Це дозволяє обробляти десятки тисяч location updates на секунду та виконувати вимогу оновлення карти не більше ніж за 5 секунд.

7. Rating Service

Приймає оцінки користувачів для ресторанів і кур'єрів та підтримує їх агрегований рейтинг. Запити надходять через API GW, після чого кожна індивідуальна оцінка надійно зберігається в *Ratings PostgreSQL*. Сервіс оновлює агреговане значення рейтингу та кешує його в Redis, щоб часто запитовані рейтинги можна було отримувати з мінімальною затримкою. Після зміни aggregate value сервіс публікує *RatingUpdated* у Kafka, звідки подія може бути передана через Realtime Gateway активним клієнтам. Компонент може бути реалізований як stateless backend service з PostgreSQL як source of truth та Redis як швидким read cache. Асинхронне поширення нового рейтингу допустиме, оскільки нефункціональна вимога дозволяє затримку до 30 секунд.

8. History Service

Надає клієнтам, кур'єрам та ресторанам доступ до завершених замовлень. Сервіс отримує *OrderCompleted* events з Kafka і формує immutable historical representation замовлення в Cassandra. У snapshot зберігаються фактичні назви страв, кількість та ціни на момент покупки, тому подальша зміна меню не впливає на старі замовлення. Історичні запити надходять через API GW та обслуговуються без навантаження на transactional *Orders PostgreSQL*. Сам сервіс може бути

реалізований як stateless API, а Cassandra - як горизонтально масштабоване NoSQL-сховище. Такий окремий read model добре підходить для постійно зростаючої історії та дозволяє оптимізувати дані під читання за customer, courier або restaurant.

Data Stores

1. PostgreSQL Restaurants & Menus

Є основним транзакційним сховищем *Restaurant & Catalog Service*. У ньому зберігаються ресторани, меню, категорії, страви, їхні описи, опції та поточні ціни. Доступ до бази має тільки сервіс-власник, а інші компоненти отримують ці дані через API або domain events. PostgreSQL є реляційною СУБД з підтримкою ACID-транзакцій, foreign keys та індексів. Це добре відповідає структурованій моделі каталогу з великою кількістю логічних зв'язків. Власна база сервісу також зменшує coupling між мікросервісами.

2. PostgreSQL Orders

Є authoritative source of truth для поточного та фінального стану замовлень. У ньому Order Service зберігає customer, restaurant, order items, фінальні ціни, статуси та необхідні timestamps. Всі зміни виконуються виключно через Order Service, а іншим компонентам інформація передається через API або Kafka events. PostgreSQL забезпечує транзакції та механізми locking/concurrency control, необхідні для коректних переходів стану. Реляційна база є доречною, оскільки order lifecycle містить критичні бізнес-операції, які не повинні частково виконуватися. Historical storage при цьому існує окремо та не замінює цю базу як source of truth.

3. PostgreSQL Payments

Зберігає внутрішній стан платіжних операцій. У базі можуть знаходитися payment identifier, order identifier, provider reference, payment status, idempotency key та timestamps, але не необроблені реквізити банківських карток. З базою безпосередньо працює лише *Payment Service*. PostgreSQL використовується через необхідність надійно фіксувати переходи між станами платежу та захищати їх транзакціями. Unique constraints можуть використовуватися для реалізації idempotency та запобігання дублюванням операціям. Такий transactional storage особливо важливий при webhook, retry та network timeout сценаріях.

4. PostgreSQL Deliveries / Assignments

Зберігає надійний зв'язок між замовленням та кур'єром, який виконує його доставку. Дані включають order_id, courier_id, статус доставки та основні timestamps, наприклад accepted, picked up і delivered. *Delivery / Dispatch Service* виконує всі операції читання і запису та використовує транзакційну або conditional update логіку для atomic assignment. PostgreSQL дозволяє застосувати unique constraints і row locking, щоб два кур'єри не могли успішно прийняти одне замовлення. Це durable storage, на відміну від Redis, який використовується лише для швидкого operational state. Збереження assignment у реляційній базі також дозволяє відновити стан доставки після перезапуску сервісу.

5. PostgreSQL Raw Ratings

Зберігає всі індивідуальні оцінки, які користувачі поставили ресторанам або кур'єрам. Записи не замінюються агрегованим значенням, тому система зберігає повну історію оцінювання та може за необхідності повторно розрахувати рейтинг. Доступ до бази має *Rating Service*, який є власником цих даних. PostgreSQL добре підходить для цієї інформації завдяки надійності запису, constraints та можливості забезпечити правило, наприклад одна оцінка на конкретну доставку. Агрегований рейтинг зберігається окремо в Redis для швидкого читання. Таким чином durable source of truth і read-optimized representation мають різні відповідальності.

6. Redis Cache Aggregated Ratings

Містить уже розраховані рейтинги ресторанів та кур'єрів, які часто читаються клієнтами. *Rating Service* оновлює значення після надходження нової оцінки, а read operations можуть отримувати рейтинг без виконання дорогої агрегації над усіма raw ratings. Redis є in-memory key-value storage, тому lookup за restaurant або courier identifier має дуже низьку latency. У разі втрати кешу дані можуть бути відновлені з *Raw Ratings PostgreSQL*, тому Redis не є source of truth. Горизонтальне масштабування Redis також дозволяє витримувати великий read traffic. Такий кеш особливо корисний, оскільки rating показується в багатьох restaurant, search та courier-related views.

7. Redis Cluster Courier State + GEO Index

Зберігає оперативний стан кур'єрів: останню позицію, availability, online status та last_seen. Location Service постійно оновлює ці дані, а *Delivery / Dispatch Service* використовує геопросторовий індекс Redis для пошуку доступних кур'єрів поблизу ресторану. *Redis GEO* дозволяє ефективно виконувати radius/distance queries без повного перебору всіх активних кур'єрів. Оскільки дані дуже часто змінюються і мають короткий життєвий цикл, in-memory storage підходить краще за традиційне durable database storage. Для stale courier state можуть використовуватися TTL або перевірка last_seen. *Redis Cluster* також дозволяє горизонтально розподілити високий потік location updates між вузлами.

8. Elasticsearch Search Index

Є спеціалізованим read-optimized індексом для пошуку ресторанів та страв. *Search Service* записує до нього документи після отримання catalog events з Kafka і використовує його для виконання користувацьких search queries. Індекс може містити назву ресторану, cuisine, назви страв, категорії, ключові слова та географічні поля. Elasticsearch підтримує full-text search, relevance scoring, fuzzy matching, фільтрацію та geospatial queries. Він не використовується як authoritative database, оскільки первинні дані ресторану знаходяться у PostgreSQL. Окремий search index дозволяє незалежно масштабувати дуже read-heavy пошукове навантаження.

9. Cassandra Historical Orders

Використовується для довготривалого зберігання історичних представлень завершених замовлень. History Service записує дані після отримання OrderCompleted event і читає їх при запитах історії через API GW. Дані можуть бути денормалізовані в окремі projections, оптимізовані під orders_by_customer, orders_by_courier та orders_by_restaurant. Cassandra є distributed wide-column NoSQL database, яка підтримує горизонтальне масштабування та високий write/read throughput. Це добре відповідає постійно зростаючому обсягу завершених замовлень та access patterns, які заздалегідь відомі. При цьому transactional *Orders PostgreSQL* залишається authoritative source, а Cassandra виконує роль масштабованого historical read model.

Messaging & Event Streaming

1. Apache Kafka

Є центральною платформою для асинхронного обміну durable domain events між мікросервісами. Через Kafka передаються події на кшталт RestaurantUpdated, MenuItemUpdated, DeliveryRequested, DeliveryOffer, CourierAssigned, RatingUpdated та OrderCompleted. Producers записують події у відповідні topics, а зацікавлені services читають їх незалежними consumer groups без прямої синхронної залежності один від одного. Kafka зберігає events на диску та підтримує повторне читання, що дозволяє consumers відновитися після тимчасового падіння. Компонент може бути реалізований як *Kafka cluster* з кількома brokers та replicated partitions. Він вибраний для критичних бізнес-подій, оскільки забезпечує durability, масштабування throughput і decoupling між сервісами.

2. Redis Pub/Sub

Використовується для швидкої передачі ephemeral real-time подій з координатами кур'єрів. Location Service публікує CourierLocationUpdated, а Realtime GW підписується на відповідні канали та пересилає нові координати через WebSocket клієнтам і ресторонам. На відміну від Kafka, *Pub/Sub* не призначений для довготривалого зберігання або replay повідомлень. Це прийнятно для GPS updates, оскільки після надходження нової координати стара позиція практично втрачає цінність. *Redis Pub/Sub* працює в оперативній пам'яті та забезпечує дуже малу затримку між publisher і subscriber. Саме тому він використовується для виконання SLA до 5 секунд, тоді як Kafka залишається каналом для durable business events.